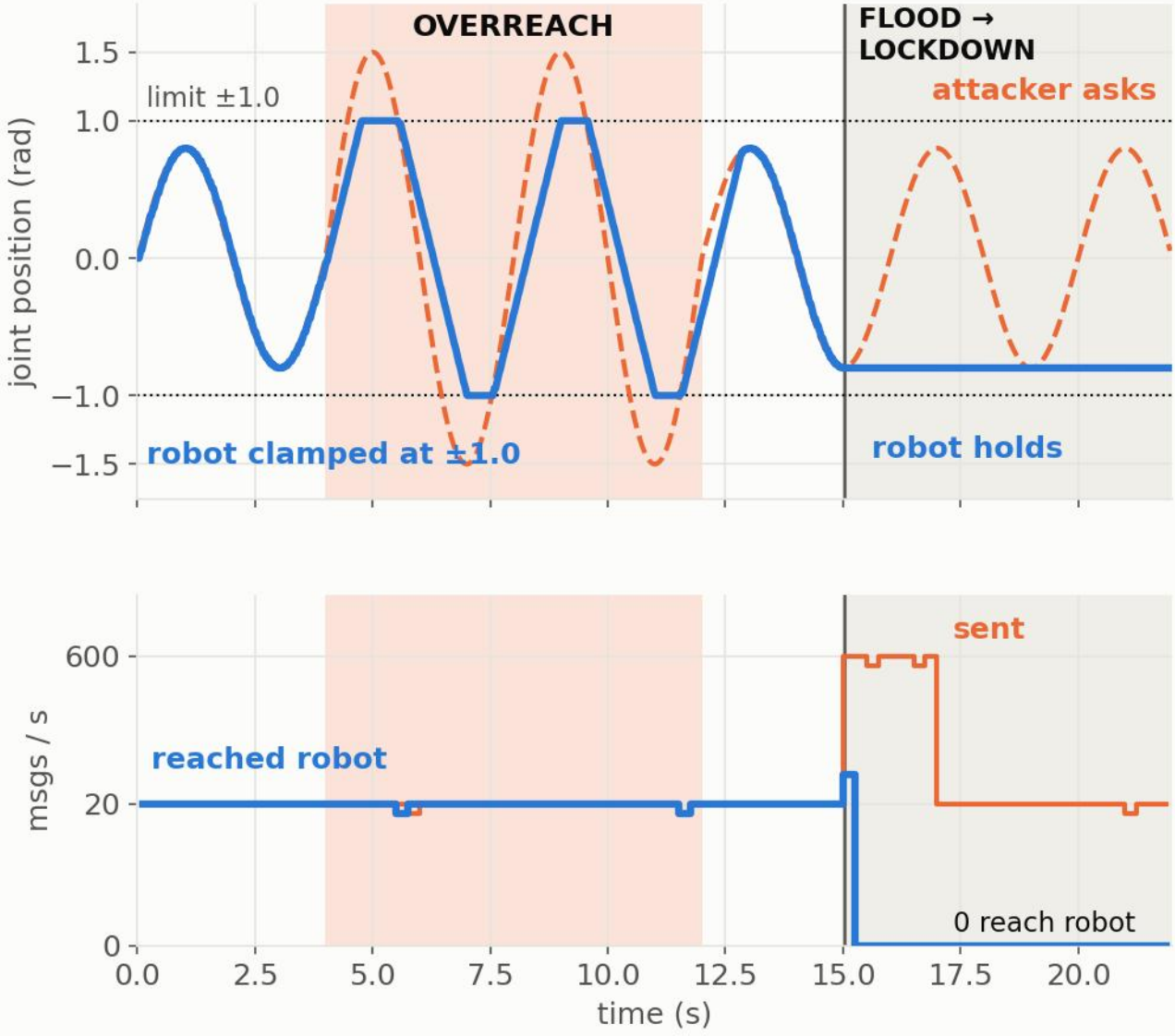
An attacker pushed the arm past its limits. It didn't move.

Live run through the Rust gateway: an overreach attack gets clamped, then a 600 msg/s flood triggers lockdown in ~10 ms.



The robot never left its safe range.

v1 watched the network.
v2 protects the robot.

RoboShield v1 parsed raw RTPS packets and raised alerts.
v2 sits between the network and the robot and acts.

Position
- **v1** passive network tap
- **v2** inline: the only path to the robot

Response
- **v1** writes an alert
- **v2** clamp, drop, or lock down

Robot safety
- **v1** none
- **v2** holds last safe pose, boots locked

Spoofed publishers
- **v1** flags DDS IDs
- **v2** drops their messages + SROS2 certs

Command content
- **v1** not inspected
- **v2** limits, jumps, NaN, replays

Robot-side network
- **v1** not covered
- **v2** rogue devices, state spoofing

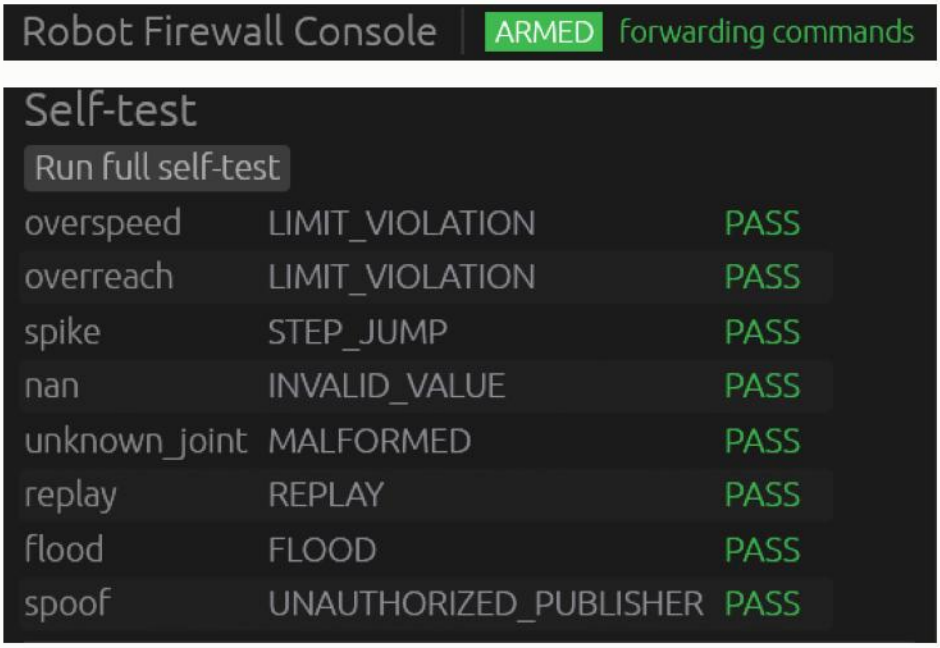
Operator tooling
- **v1** command line
- **v2** GUI: every command is a button

Same project, one level up: from detecting to preventing.

github.com/Amin-Ahmed-G/robotshield

8 attack types.
8 caught. One click.

The Rust console runs every attack against the live firewall and checks each one is reported. Also 8/8 in a run with SROS2 enforced.



Status bar and self-test panel from the console, after one run.

8 / 8

attacks detected

1.000

max robot position (limit 1.0 rad)

0

msgs through from a cert-less attacker*

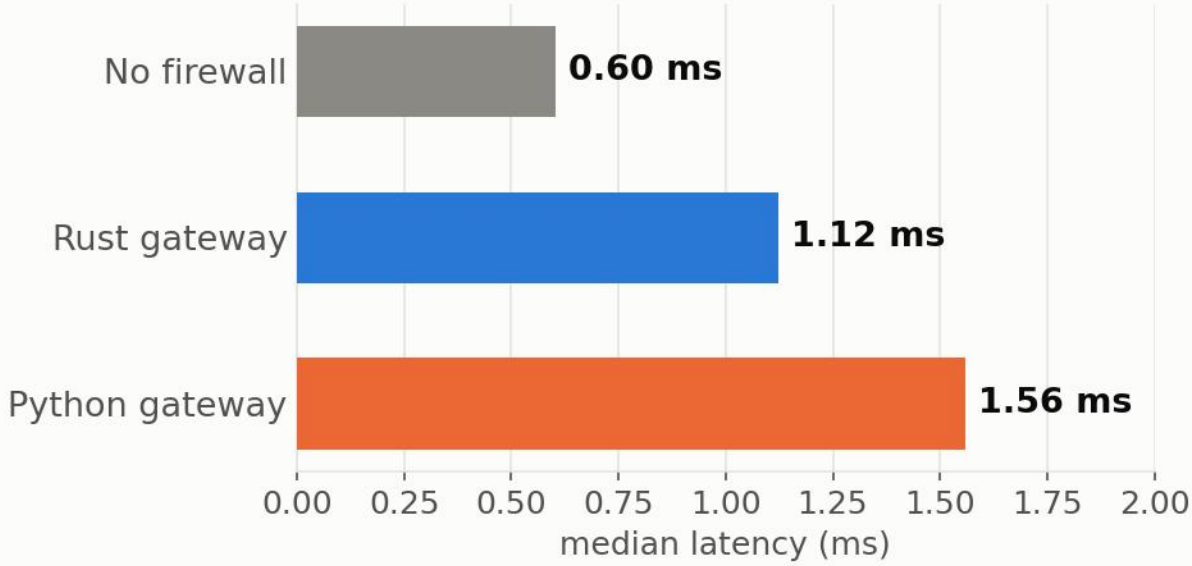
* SROS2-enforced run, attacker publishing without certificates throughout.

Every debug command is a button. No terminal juggling.

github.com/Amin-Ahmed-G/robotshield

Security that costs 0.5 ms per command.

Median time for a command to go from the laptop to the robot, with no firewall, through the Rust gateway, and through Python.



1.37 µs

to check one command (7x faster than Python)

1000/1000

commands delivered in every latency run

Laptop benchmark (Ryzen 7 7435HS, ROS 2 Jazzy, Fast DDS). NanoPi numbers next.

Fast enough for real-time control loops.

github.com/Amin-Ahmed-G/robotshield